

Entregable 2

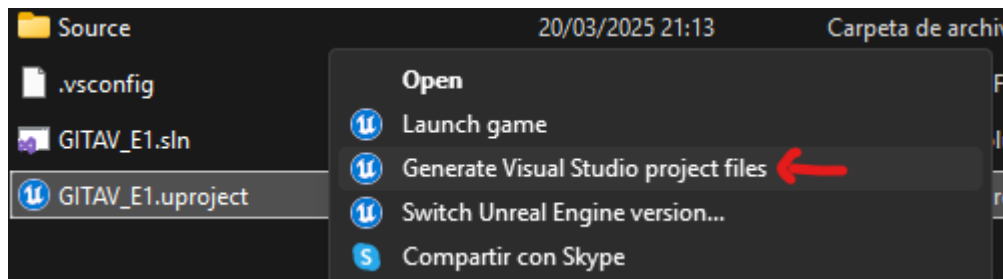
Conceptos previos:

El entregable consta de un proyecto de Unreal Engine (v 5.6) que deberás completar correctamente, en su mayoría usando código en lenguaje C++.

Para ello, descarga del campus virtual (Blackboard) el archivo **Entregable2_Base.zip**

Crea una carpeta con tu nombre y apellidos con este formato "**NombreApellido1Apellido2**" y descomprime el contenido del paquete en ella.

En primer lugar, genera los archivos de Visual Studio así:



Ahora, abre el archivo .sln y asegúrate que funciona todo correctamente. Deberías ser capaz de compilar correctamente la Solución y de abrir el motor desde Visual Studio.

Reglas a seguir:

Al abrir el proyecto verás una serie de niveles. El nivel sobre el que has de trabajar es el llamado **E02**.

Para completar al 100% esta entrega, debes realizar los 3 ejercicios que se indican a continuación.

El ejercicio 2 es **obligatorio** hacerlo en C++. Después, puedes utilizar las funciones como nodos en Blueprints.

Para el ejercicio 3 puedes utilizar C++ y Blueprints combinados o únicamente Blueprints. La forma en que hagas esto **no modificará tu puntuación** por lo que siéntete libre de usar el sistema que prefieras.

Reglas generales:

En el Editor:

- Cada objeto debe estar situado en el nivel correcto. Esto es, si creas un Blueprint que contiene lógica de juego, debe estar situado en el nivel GMP.
- Tienes a tu disposición en la carpeta “Art” los recursos necesarios para vestir correctamente los assets que faltan (ver ejercicio 3)
- **El juego debe ejecutarse y compilar sin errores.** En caso contrario, el entregable computará como entregado pero con una puntuación de 0.

Descripción general y objetivo del entregable:

El objetivo de esta entrega es poner en práctica lo aprendido hasta ahora en la asignatura. Con especial interés en la metodología de carga y descarga de niveles de manera eficiente (level streaming).

Partirás de un proyecto incompleto donde todos los actores están contenidos en un único nivel. Debes realizar el trabajo de un diseñador de niveles y dividir el juego por zonas según las imágenes que verás en los ejercicios.

También, debes configurar correctamente la carga y descarga de niveles, y por último has de implementar un flujo de juego para ofrecer 3 desafíos al jugador (1 en cada habitación) y poder guiarle para que termine el nivel.

Para hacer esto, debes crear los objetos que consideres necesarios, pero recuerda situar estos objetos en el nivel adecuado.

Ejercicio 1. Distribución de estructura de niveles (3 puntos)

En este ejercicio debes acceder al mapa persistente **E02** y crear una estructura de niveles adecuada que se adapte al diseño del nivel de la imagen:



El nivel del recuadro verde corresponde a la habitación 1, el rojo a la habitación 2 y el amarillo a la habitación 3.

La X roja indica el PlayerStart. La X verde indica la estatuilla que marca el final del nivel.

Por tanto, deberás crear 3 subniveles y cada uno de ellos deberá tener sus respectivos niveles de Gameplay, Arquitectura, etc. Se detallan a continuación:

Nivel Habitación 1 contiene estos subniveles:

- Gameplay (GMP)
- Arquitectura (ARCH)

Nivel Habitación 2 contiene estos subniveles:

- Gameplay (GMP)
- Arquitectura (ARCH)

Nivel Habitación 3 contiene estos subniveles:

- Gameplay (GMP)
- Arquitectura (ARCH)
- Iluminación (LIGHT)

Tú mismo debes distribuir los assets como creas conveniente en cada una de las habitaciones pero debes seguir estas **reglas obligatorias**:

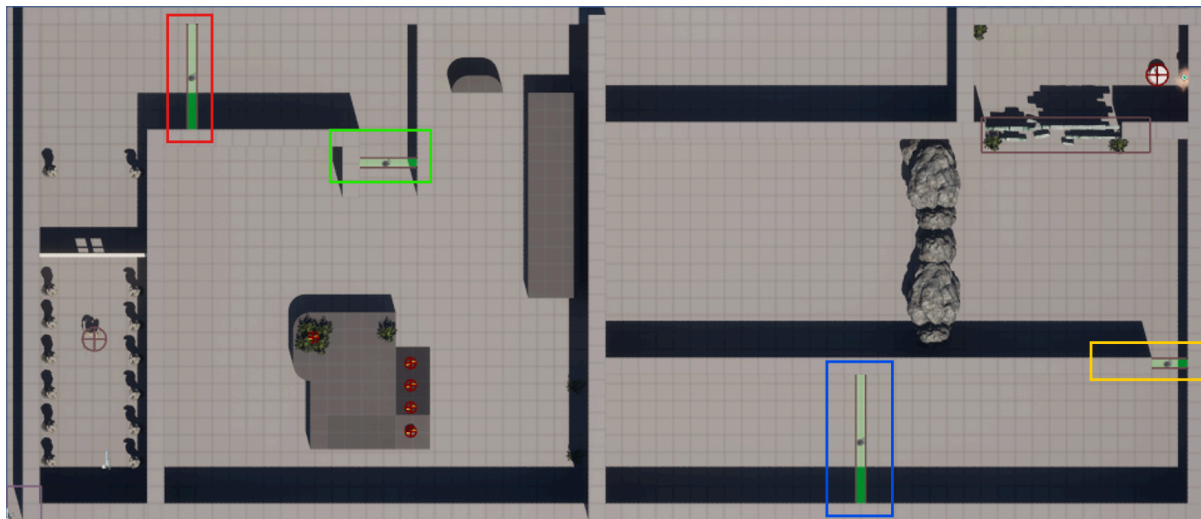
- **La primera habitación está cargada desde el inicio (ver vídeo).**
- **Las habitaciones 2 y 3 están descargadas desde el inicio.**

- Los niveles de Gameplay no deben contener elementos meramente decorativos (para eso está el nivel ARCH) y viceversa.
- Debes decorar los niveles adecuadamente. Esto puedes hacerlo a tu gusto, puedes seguir las ideas de las capturas de pantalla de este documento (es igual que en los videos) o puedes vestir el nivel como tú prefieras. Pero es obligatorio poner “props” en cada una de las habitaciones para poder comprobar el funcionamiento correcto del streaming.

Ejercicio 2. Streaming de niveles (4 puntos)

Este ejercicio debe realizarse de manera obligatoria en C++.

Crea funciones en C++ para cargar y descargar niveles de manera asíncrona y vincula su comportamiento con los triggers indicados en la imagen, de manera que cumplan las siguientes funciones:



Trigger 1 (en rojo) - Carga habitacion 2

Trigger 2 (en verde) - Descarga habitación 1

Trigger 3 (en azul) - Carga habitación 3, cierra puerta movable (ver ejercicio 3)

Trigger 4 (en amarillo) - Descarga habitación 2

Una vez hayas creado estas funciones, utilízalas en Blueprints de nivel para realizar la carga / descarga eficiente de niveles una vez se atraviese un trigger en concreto.

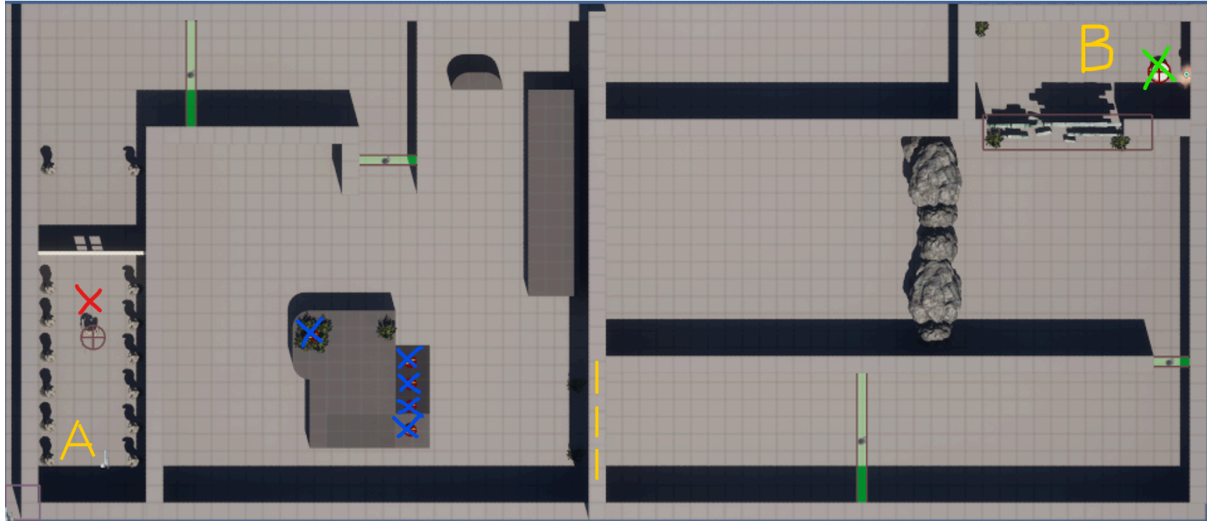
La idea del streaming de niveles es que el jugador no note cambios en la escena mientras juega. Por lo tanto, asegúrate de que “camuflas” el streaming correctamente.

Por razones obvias, ten cuidado con descargar el suelo del nivel en el que estás. Tenlo en cuenta también para las paredes.

Ejercicio 3. Lógica de juego. Creando un Gameplay (3 puntos)

Implementa la lógica y el flujo necesario para interactuar con el nivel correctamente y poder guiar al jugador hasta el final.

Siguiendo la imagen, el jugador debe ir desde el punto A hasta el punto B pasando por las 3 estancias.



Para poder avanzar correctamente, el jugador ha de ir completando una serie de tareas que se detallan a continuación:

Para completar la habitación 1:

- El jugador debe derrotar al enemigo que custodia la puerta (X roja). Cuando lo hace, esta se abre automáticamente, permitiendo avanzar al jugador.

Para completar la habitación 2:

- El jugador debe encontrar las 5 monedas (X azules). Cuando ha obtenido todas, y solo en este caso, se abrirá una pared secreta que permitirá acceso a un pasillo, que conduce a la última estancia.

Para completar la habitación 3:

- El jugador debe localizar la pared rompible y utilizar sus habilidades de combate para derribarla. Le costará unos cuantos golpes y cuando lo consiga, la puerta se destruirá, abriendo paso a una estancia secreta. Aquí, encontrará una estatuilla (X verde) bajo la luz de una antorcha. El jugador debe **interactuar** con ella para completar el nivel.

Tu misión en este ejercicio es acomodar el gameplay correctamente para que el jugador pueda llegar hasta el final. Configurando de manera adecuada todos los desafíos y creando una experiencia como la del vídeo.



Para realizar esto, debes crear todos los Blueprints que consideres necesarios (recuerda, puedes usar tanto C++ como Blueprints, o una combinación de ambas, pero no es obligatorio usar código C++).

Sin embargo, sí es obligatorio cumplir las siguientes reglas:

- El jugador debe poder **Golpear** otros actores (Tecla F)
- El jugador debe poder **Interactuar** con la estatuilla final (Tecla E)
- Es **obligatorio** el uso de interfaces (Blueprint Interface o Interfaces en C++, queda a tú elección)
- El **enemigo** debe tener una barra de vida y bajar conforme es golpeado por el personaje. Cuando muere, el actor del enemigo se destruye.

Requisitos opcionales:

En el **ejercicio 3**, puedes realizar cualquiera de estas tareas para obtener más puntos. Estas tareas no son obligatorias, pero hacerlas te hará subir la puntuación del ejercicio.

Tareas (cada una de ellas sumará 0.2 puntos al ejercicio 3):

- Abre las puertas dobles de la habitación 1 de forma animada.
- Abre la puerta deslizante de la habitación 2 de forma animada.
- Crea un HUD en el jugador para llevar el recuento de las monedas acumuladas.
- Crea un widget para la pared rompible que indique cuanta vida tiene.
- Crea una moneda grande de color azul que proporcione 5 monedas extra a la cuenta.

Consejos generales:

Conforme vayas realizando los ejercicios, te darás cuenta de que todos están vinculados entre sí. No podrás hacer el streaming de niveles si no has dividido los niveles correctamente por lo que ten en cuenta esta circunstancia a la hora de hacer el ejercicio 2.

Importante: Debes tener en cuenta que un nivel que no está cargado no existe a nivel de ejecución. También es importante saber que un evento **Begin Play** de un nivel descargado no tiene lugar hasta que dicho nivel no ha terminado de cargarse.

¿Qué debo subir a Blackboard?

Cuando hayas terminado, tu proyecto debe estar contenido en la carpeta inicial que creaste llamada "**NombreApellido1Apellido2**". Comprime esa carpeta (formato .zip o .rar a ser posible) y llama al archivo final de igual manera (debe llamarse **NombreApellido1Apellido2.zip**)

Sube ese archivo comprimido a Blackboard a la parte correspondiente de la actividad del entregable.